MORPH-DA: A Mutation-Grounded Benchmark for Metamorphic Verification of Data-Analysis Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

Data-analysis agents increasingly generate and execute Python programs, but successful execution does not establish that the program implemented the requested filters, aggregation, grouping, denominator, or date window. We introduce MORPH-DA, a mutation-grounded benchmark for evaluating runtime verifiers of data-analysis agents, together with a specification-conditioned metamorphic reference verifier for *wrong-but-executable* programs. The reference verifier keeps each generated program fixed, applies operator-conditioned transformations to the underlying tabular environment, and checks necessary output relations without using a precomputed numeric gold answer or an additional model call. The benchmark contains 101 compositional tasks and 563 validated, non-equivalent semantic mutants. MORPH-DA detects 364/563 faults (64.7%, 95% task-clustered CI [58.4, 70.5]) versus 9/563 for universal robustness checks ($p = 9.4 \times 10^{-79}$). Re-executing each seed-42 agent program unchanged on held-out data seeds reveals that 13–16 programs per model (20–26% of apparently correct programs) are *accidental corrects*: correct on the source seed but wrong on at least one held-out seed. With cross-seed evaluation labels, MORPH-DA obtains 79–89% precision and 67–79% recall on executable programs, with 8–21% false-positive rate. These results position metamorphic execution as a complementary, zero-additional-LLM-call verifier rather than a correctness certificate.

1 Introduction

LLM-powered data-analysis agents translate natural-language questions into executable programs over tabular data. An execution monitor catches syntax errors, missing columns, and runtime exceptions, but a program can run cleanly while performing the wrong analysis: omitting a segment filter, using a sum instead of a mean, counting rows instead of distinct entities, grouping by the wrong dimension, or swapping current and comparison periods. We call these *wrong-but-executable programs* (WEPs). Their outputs are especially dangerous because they are plausible, reproducible, and accompanied by no runtime signal.

Gold-answer evaluation identifies WEPs in a benchmark, but the correct output is rarely available during deployment and gives little diagnostic information for repair. A second problem is less visible: a structurally wrong program can coincidentally return the correct answer on one finite dataset. For example, omitting a cancelled-order filter may not change the top category in one data realization but can change it after the category mix shifts. We call such cases *accidental corrects*. They make single-instance accuracy an optimistic estimate of whether an agent implemented the requested computation.

MORPH-DA addresses these problems through metamorphic runtime verification. Instead of asking whether the original answer matches an unavailable gold output, it asks whether the same generated

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

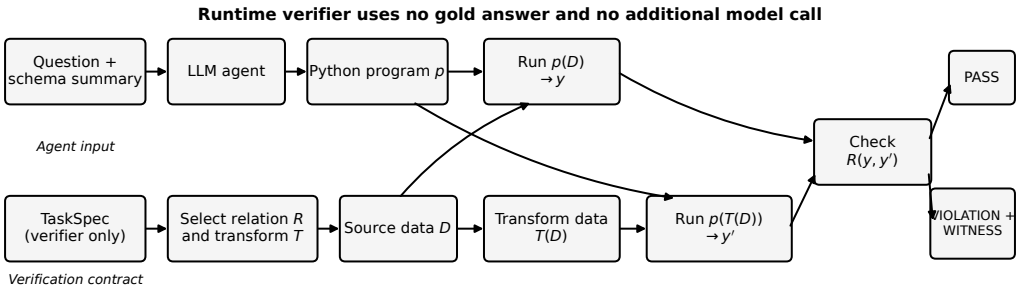

Figure 1: MORPH-DA separates program generation from verification. The agent receives only the question and schema summary. The verifier receives the candidate program and a structured task specification, generates deterministic follow-up tables, re-executes the unchanged program, and checks the expected output relation. No gold answer or additional LLM call is used during verification.

program behaves predictably after controlled changes to the data. Adding extreme rows outside a requested date range should not change the answer; duplicating all rows should double a sum but preserve a mean; perturbing a non-median extreme should change a mean but not a median; and constructing a dominant counterfactual group should force the expected winner. A violated relation produces a concrete counterexample witness.

Our contributions are:

1. **A mutation-grounded verification benchmark.** We provide 101 compositional analytical tasks and 563 validated non-equivalent mutants across aggregation, filtering, grouping, ranking, and hardcoding faults.
2. **Operator-aware, environment-grounded verification.** More than 20 relations transform the underlying data and check invariance, equivariance, monotonicity, or a forced counterfactual outcome without exposing the gold answer.
3. **An accidental-correct evaluation protocol.** We hold each generated program fixed and re-execute it on held-out data distributions, revealing a substantial gap between single-seed and structural correctness.
4. **Controlled and natural-agent evidence.** We report mutation coverage, natural-program precision/recall and accepted-answer risk, multi-transformation sensitivity, and a paired one-step repair study.

2 Related Work

Data-analysis agents and benchmarks. InfiAgent-DABench evaluates end-to-end analysis over CSV files [Hu et al., 2024]. DS-1000 and DSCodeBench test data-science code generation [Lai et al., 2022, Ouyang et al., 2025]. Recent benchmarks broaden coverage to agentic workflows, heterogeneous workspaces, and end-to-end computer environments [Zhang et al., 2025, Nam et al., 2026, Li et al., 2026, Sun et al., 2026, Rahman et al., 2026]. These works primarily score final task success. MORPH-DA instead evaluates whether an execution-grounded verifier detects semantic faults, including programs that happen to return the correct output on one dataset.

Metamorphic and mutation testing. Metamorphic testing validates necessary relationships across transformed inputs when a direct test oracle is unavailable [Segura et al., 2016]. Mutation testing injects controlled faults to quantify a test suite's sensitivity [Jia and Harman, 2011, Petrović et al., 2021]. Prior work applies metamorphic transformations to text-to-SQL systems and language models [Ma and Wang, 2020, Yang et al., 2025, Cho et al., 2025]. MORPH-DA differs in three ways: it transforms the *tabular data environment* while keeping the candidate program fixed (rather than mutating the query or prompt); it activates relations only when the task specification warrants a particular analytical operator (e.g., a duplication test fires only for sum or mean metrics, not for every program); and it uses a validated semantic-mutant corpus to quantify how much of the fault space each relation family covers. This last property is consequential: a program using the wrong

aggregation function can coincidentally return the correct ranking label on one dataset, and mutation scoring measures the fraction of such faults a verifier can detect via data-state transformations alone.

Verifier meta-evaluation. Recent work increasingly treats verifiers themselves as objects of evaluation. Dücker et al. introduce a benchmark for meta-evaluating agentic verifiers over rubric-graded, heterogeneous deliverables [Dücker et al., 2026]. Ma et al. develop an execution-based agentic verifier that searches for discriminative test inputs exposing behavioral differences among candidate programs [Ma et al., 2026]. Wang and Zhu validate LLM-generated programs by varying the prompt and cross-checking consistency among independently generated implementations [Wang and Zhu, 2024]. MORPH-DA differs by keeping a single candidate analysis program fixed, applying specification-conditioned transformations to its tabular execution environment, and measuring verifier coverage against validated semantic mutants.

Verification signals for agents. Model-based judges and self-critique can inspect a trace, but they introduce an additional model call and may share the generator's semantic blind spots [Wang and Zhu, 2024]. MORPH-DA supplies a deterministic, falsifiable signal: a specific transformed dataset and pair of outputs that violate a declared relation. It is complementary to model-based judging rather than a replacement for it.

3 Problem Setting

A task is $t = (q, D, s)$, where q is a natural-language question, D is a dictionary of DataFrames, and s is a structured analytical specification. The agent sees q and a schema summary, and produces a reusable program p implementing `analyze(tables)`. The verifier sees p and s, but not a reference program or gold output.

The source result is $y = p(D)$. A metamorphic test $m = (T, R)$ contains a deterministic data transformation T and a necessary output relation R. It produces $D' = T(D)$ and $y' = p(D')$. A violation is

$$v_m(p, t) = \mathbf{1}[\neg R_m(y, y')] \,. \tag{1}$$

The program is flagged if any applicable relation violates. Relations express equality, known scaling, monotonic change, or a forced winner. Passing all relations is not proof of correctness: each relation encodes only a necessary condition.

For controlled evaluation, a semantic mutant is *killed* when at least one relation flags it. Mutation score is the fraction of validated, non-equivalent mutants killed. For natural programs, we report precision, recall, false-positive rate (FPR), and accepted-answer risk $\mathrm{AAR} = \mathrm{FN}/(\mathrm{FN} + \mathrm{TN})$, the probability that an accepted executable program is wrong.

Trust model. The analytical contract encoded in s and the transformation engine are assumed trusted; the agent-generated program p and its output y are untrusted. The runtime verifier does not consume a precomputed numeric answer: it checks only necessary behavioral properties. In the controlled benchmark, the complete `TaskSpec` is compilable, so we do not claim that metamorphic checks dominate trusted reference execution when such an implementation is available. The intended deployment setting is one where governed analytical properties are available but maintaining an independently trusted reference implementation for every generated analysis is impractical. The method is gold-answer-free at runtime, not supervision-free. MORPH-DA verifies the executable analytical artifact produced by the agent, not the agent's planning trace or tool-selection policy.

4 MORPH-DA

4.1 Specification-conditioned relation library

Each `TaskSpec` records filters, metric operators, grouping dimensions, date windows, ranking direction, comparison periods, and output type. The natural-language question is generated from this specification, but the agent never receives the specification itself. This separation lets the benchmark test whether an agent inferred and implemented the requested computation, while the verifier checks compliance without knowing the numeric answer.

Relations are activated only when their premises are justified by the specification. Representative families are:

- **Universal robustness:** row permutation and column-order invariance detect accidental positional dependence.
- **Filter and scope:** inject extreme rows that violate a required filter; a correct program must ignore them. A complementary in-scope sentinel can be constructed to force a winner.
- **Aggregation and statistics:** full duplication preserves means, medians, distinct counts, ratios, and rankings but scales sums and counts. Non-median extreme perturbations distinguish mean from median. Translation, scaling, and count-vs-distinct checks provide additional operator-specific tests.
- **Grouping and ranking:** counterfactual group construction, rank reversal, and top-k consistency expose wrong grouping dimensions, ascending/descending errors, and hardcoded labels.
- **Time and period comparisons:** period-isolated injections and a forced year-over-year winner expose swapped periods, missing date windows, and absolute-vs-relative change errors.

The current library supports sum, mean, median, count, distinct count, min, max, standard deviation, variance, quantile, ratio, percentage change, and correlation. Fields for weighted means and joins exist in the specification DSL, but no weighted-mean or join relations are implemented or evaluated in the present corpus.

4.2 Counterexample witnesses and repair

When a relation fails, the verifier records the transformation, source output, follow-up output, expected relation, and likely fault family. This record is a concrete counterexample rather than an opaque confidence score. In the repair experiment, the same base program is paired across four feedback strategies: no retry, generic feedback, the violated relation name, and a full witness.

4.3 Multi-transformation sensitivity

Many transformations contain randomized but deterministic choices, such as which eligible rows become sentinels. Running a second verifier random seed generates a different set of follow-up tables while keeping the candidate program and source data fixed. We use this as a sensitivity analysis, not as a uniformly preferred deployment mode, because additional tests can increase both recall and false positives.

5 Benchmark Construction

Tasks and data. The benchmark contains 101 single-table tasks over eight synthetic business scenarios: retail orders, web sessions, seller marketplace, SaaS subscriptions, marketing campaigns, payments, fulfillment, and customer support. Tasks span five compositional levels: scalar aggregation (24), grouped ranking (48), ratios or support thresholds (11), year-over-year comparison (16), and multi-filter ratio-plus-comparison tasks (2). Data generators include heavy-tailed metrics, nulls, duplicates, and current/prior periods. The same schema and question are retained while seeds change the underlying distribution.

Filter discriminability. For each required filter, we verify that applying it changes the gold answer on seeds 7, 42, and 123. Three tasks fail this discriminability check and are excluded from the 98-task cross-seed evaluation set; otherwise, a program omitting the filter could pass regardless of implementation.

Reference compiler and consistency. A structural compiler translates each `TaskSpec` into a Pandas reference program. All references execute on seeds 7, 42, and 123 and pass applicable relations. Because references and relations are co-developed from the same specification, this establishes benchmark internal consistency, not an independent zero-FPR estimate. Independent false-positive behavior is measured on LLM-generated programs.

Table 1: Detection on 563 validated, non-equivalent mutants. The intermediate configuration is a genuine run recorded alongside the full result.

Verifier	Killed	Score	95% CI
Execution only	0	0.0%	–
Universal relations	9	1.6%	–
Universal + filter + aggregation	346	61.5%	–
Full MORPH-DA	**364**	**64.7%**	**[58.4, 70.5]**

Mutation funnel. AST operators generate 3,030 candidate mutations. Applicability, syntax, execution, and output-contract filters leave 709 candidates for equivalence testing. Across five oracle seeds, 146/709 (20.6%) are equivalent and removed, yielding 563 validated non-equivalent mutants: 131 aggregation, 173 filter, 21 grouping, 129 hardcoding, and 109 ranking faults.

6 Experiments

Agents. A LangGraph code agent generates one Python program per task from the question and schema summary using Claude Haiku 4.5, Sonnet 4.6, or Opus 4.5. Evaluation centers on the 101 seed-42 programs per model. The framework is an implementation substrate, not part of the claimed method.

Three evaluation conditions. All three conditions use the same 98 filter-discriminating tasks. Condition A uses naive seed-42 gold labels (treating accidental corrects as correct). Condition B saves each seed-42 program and re-executes that exact source unchanged on seeds 7 and 123; failure on either reclassifies it as wrong (cross-seed evaluation labels). Condition C retains the same cross-seed ground truth but runs MORPH-DA with two transformation seeds instead of one. Precision, recall, and FPR exclude original execution failures; their counts are reported separately. For reference, Condition A on the full 101-task set is reported in Appendix A.1.

Statistics. We use continuity-corrected McNemar tests for paired detector comparisons, Holm correction across the three natural-agent comparisons, and a task-clustered bootstrap with 3,000 resamples for confidence intervals. The mutation confidence interval also clusters by base task to avoid treating multiple mutants of one task as independent.

7 Results

7.1 Controlled semantic faults

Table 1 shows that execution detects none of the semantic mutants, while universal row/column robustness checks kill only 9. Operator-conditioned relations account for the gain: full MORPH-DA kills 364 mutants ($\chi^2 = 353$, $n_{01} = 355$, $n_{10} = 0$, $p = 9.4 \times 10^{-79}$). Filter and aggregation relations produce 346 kills; grouping, hardcoding, time, and statistical relations add 18. Thus the approximately $40\times$ ratio over universal checks primarily quantifies how weak non-semantic robustness tests are for operator faults.

Detection varies by family (Figure 2): hardcoding 85.3%, grouping 81.0%, ranking 76.1%, filtering 67.6%, and aggregation 28.2%. Low aggregation coverage is partly unidentifiability: if both sum and mean select the same top group, no output-only relation can distinguish them on that instance.

7.2 Accidental correctness

On seed 42, 61/101 Haiku, 65/101 Sonnet, and 62/101 Opus programs appear correct. Re-executing those same programs unchanged on held-out seeds reduces cross-seed-correct counts to 45, 52, and 49 (Figure 3). Therefore, 16, 13, and 13 programs – 26.2%, 20.0%, and 21.0% of apparent successes – are accidental corrects: correct on seed 42 but wrong on at least one held-out seed. Without seeing held-out seeds, single-seed MORPH-DA catches 13/16 (81%), 7/13 (54%), and 10/13 (77%) of them.

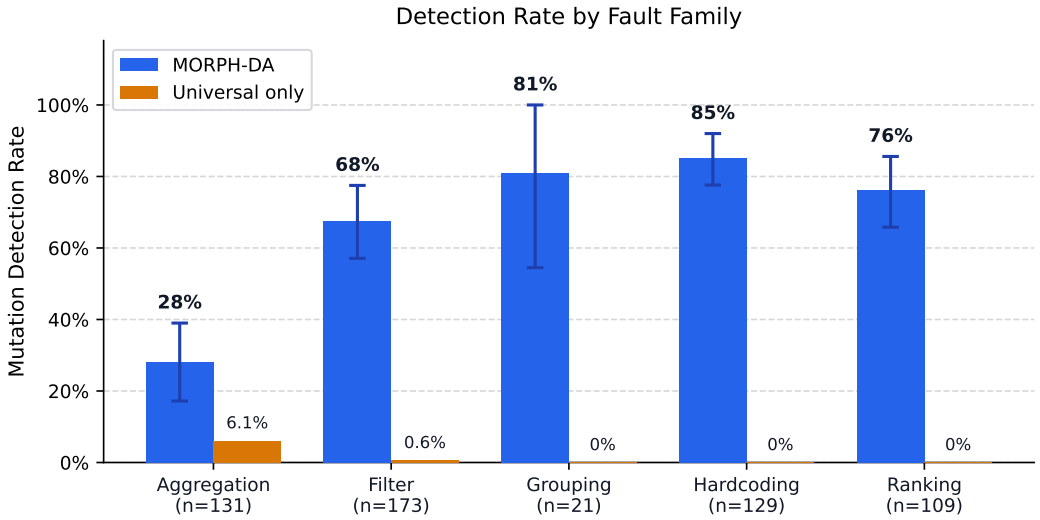

Figure 2: Mutation detection by fault family. Aggregation is hardest because a wrong aggregation can preserve the winning label on ranking tasks. Error bars are task-clustered 95% bootstrap intervals.

Table 2: Natural-program verification. All three conditions use the same 98-task filter-discriminating set and exclude original execution failures (Haiku 2, Sonnet 0, Opus 2) from denominators. Condition A uses naive seed-42 gold labels; Conditions B/C use cross-seed evaluation labels from fixed-program re-execution on seeds 7 and 123. CIs shown for the primary Condition B. 101-task naive results (Condition A-101) are reported in Appendix A.1.

Model	Condition	Precision	Recall	FPR	F1	AAR
Haiku 4.5	A: naive (98 tasks)	60.4	78.4	32.2	68.2	16.7
	B: cross-seed	**87.5** [77.1,95.8]	**79.2** [67.9,90.6]	**14.0** [4.7,25.6]	83.2	22.9
	C: two transformations	86.0	81.1	16.3	83.5	21.7
Sonnet 4.6	A: naive (98 tasks)	70.3	72.2	17.7	71.2	16.4
	B: cross-seed	**89.2** [78.4,97.3]	**67.3** [53.1,79.6]	**8.2** [2.0,18.4]	76.7	26.2
	C: two transformations	86.8	67.3	10.2	75.9	26.7
Opus 4.5	A: naive (98 tasks)	58.3	80.0	32.8	67.5	14.6
	B: cross-seed	**79.2** [68.8,89.6]	**79.2** [66.7,89.6]	**20.8** [10.4,33.3]	79.2	20.8
	C: two transformations	78.0	81.2	22.9	79.6	19.6

Manual inspection of the accidental-correct programs reveals three recurring failure modes. The dominant pattern is a **missing status or eligibility filter**: the program omits or substitutes the required row-level condition (e.g., applying `notna()` as a proxy for `status != 'open'`, or filtering for `status != 'returned'` when `status != 'cancelled'` is required), and the excluded segment happens not to shift the ranking winner on the seed-42 distribution. A second pattern is **elided date scope**: the program skips the required date window entirely and the dominant group is the same across historical data as within the target period. A third pattern appears in year-over-year and quarterly tasks: the program implements the correct temporal structure but omits a required status filter; because the excluded segment is proportionally similar across both periods on seed 42, the ranking winner is preserved. MORPH-DA detects the first two categories primarily through MR-F1 (filter injection) and MR-T (time scope) relations; the third is harder to detect because the filter omission is symmetric across periods and produces no observable difference under single-seed transformation.

7.3 Natural-program verification

Under naive labels on the same 98-task set, correctly flagged accidental programs are counted as false positives, depressing apparent precision to 58–70%. With fixed-program cross-seed ground

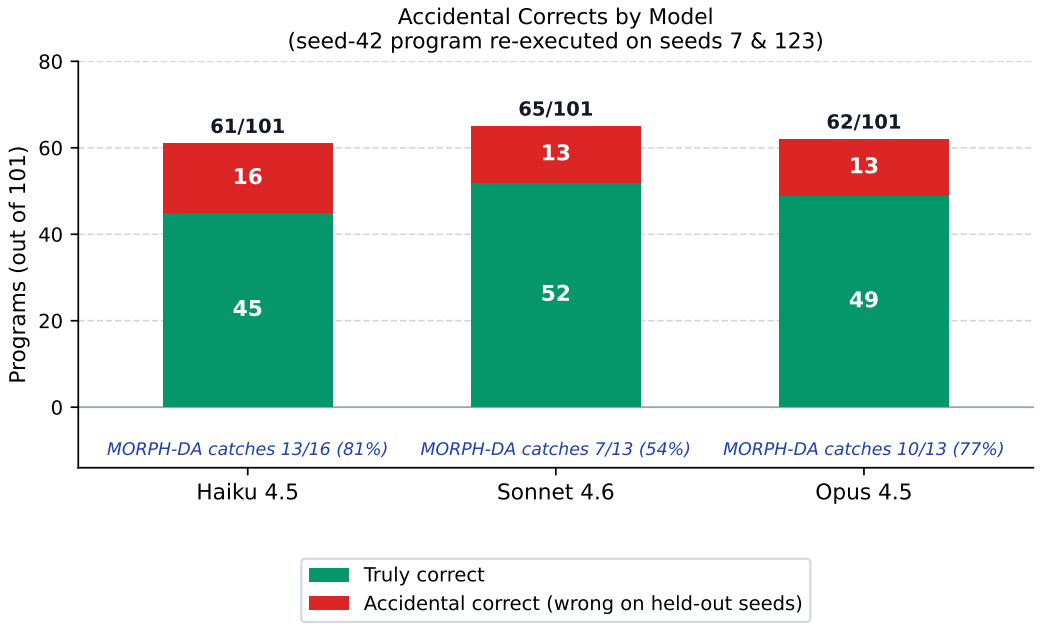

Figure 3: Programs correct on seed 42 split into cross-seed correct (correct on all evaluated seeds) and accidental correct (correct on seed 42 but wrong on seed 7 or 123) after fixed-program re-execution.

Table 3: Paired one-step repair on 91 wrong-but-executable Haiku programs from seeds 7, 42, and 123.

ID	Feedback	Fixed	Rate
R0	No retry	0/91	0.0%
R2	Generic "has a bug"	5/91	5.5%
R6	Violated relation name	11/91	12.1%
R7	Full counterexample witness	11/91	12.1%

truth, Condition B obtains 79–89% precision and 67–79% recall (Table 2). Against universal-only relations, the full verifier adds 37–48 additional flags and loses none ($n_{10} = 0$ for all models; Holm-corrected $p \leq 3.25 \times 10^{-9}$). These additional flags are predominantly true positives on wrong programs: TP counts are 33, 42, and 38 for Sonnet, Haiku, and Opus respectively. False positives remain nontrivial: 8.2–20.8%, driven mainly by extreme filter injections, tie-sensitive forced-group tests, and legal computed constants flagged by the hardcoding relation.

A second transformation seed changes recall by +1.9, 0.0, and +2.1 percentage points for Haiku, Sonnet, and Opus, while increasing FPR by about 2 points for each model. It slightly improves F1 for Haiku and Opus but lowers it for Sonnet. Condition C is therefore a multi-transformation sensitivity analysis rather than a default; deployments that value recall over false alarms may choose it.

7.4 One-step repair

The 91-program sample spans seeds 7 (25 programs), 42 (38), and 123 (28), covering 55 distinct tasks. Relation-name and witness feedback each produce 11/91 fixes versus 5/91 for generic feedback (Table 3). For each comparison with R2, $n_{01} = 6$ and $n_{10} = 0$ ($p = 0.041$ unadjusted; $p = 0.082$ after Holm correction for two co-primary comparisons), so we report a higher observed rate rather than a superiority claim. R6 and R7 have the same aggregate rate but repair different programs ($n_{01} = 4$, $n_{10} = 4$, $p = 0.29$); the study does not establish that a richer one-step witness is better than the relation name alone.

Runtime cost. Verification adds no model calls. In a 20-task reference-program pilot measured with raw Python timer timestamps, the full relation set required approximately 40 Python executions per task and had 500 ms mean, 525 ms median, and 1.14 s p95 runtime. This is a small pilot, not a production latency claim.

8 Limitations and Discussion

Specification dependence. MORPH-DA assumes a structured analytical specification. Such specifications may be authored by analysts, derived from governed metric definitions, or extracted by a separate model; automatic extraction and verification of the specification itself are outside scope. The method is gold-answer-free, not supervision-free.

Incomplete necessary conditions. Passing the current relations does not prove correctness. Aggregation coverage is especially limited on label-output tasks when wrong operators preserve the winner. New relation families, richer output contracts, and combinations with learned judges may improve recall.

Benchmark and model scope. The evaluated data are synthetic, all tasks are single-table, and all natural programs come from three variants of one model family using a LangGraph harness. Join and weighted-mean fields are represented in the DSL, but neither capability has implemented relations or evaluated tasks. Generalization to open-weight models, SQL, multi-table analysis, and real enterprise datasets remains to be tested. We note that the task structures and analytical operators—scalar aggregation, grouped ranking, period-over-period comparison, and filtered ratio—reflect query patterns common in enterprise analytics workflows. The synthetic generator and task set were finalized before natural-agent evaluation began; observed LLM failure modes were not used to tune task difficulty or filter coverage, avoiding circularity between task design and measurement.

Reference and repair limitations. Reference programs and relations share the same specifications, so reference pass rates measure internal consistency rather than an independent FPR. Natural-agent programs provide the independent FPR estimate. Finally, repair success is low and one-step only; equal aggregate R6/R7 rates and non-significant paired differences motivate multi-round and fault-specific repair studies.

Implications for agent verification. The main lesson is not that metamorphic testing replaces gold evaluation. Instead, it supplies a reusable environment-grounded signal where direct answers are unavailable, exposes single-instance accidental correctness, and creates counterexamples that can support debugging, model-based judging, or human review.

9 Conclusion

Execution success is an inadequate verifier for analytical agents, and single-instance correctness can conceal structurally wrong programs. MORPH-DA combines a mutation-grounded benchmark with specification-conditioned data transformations to test necessary behavioral properties without revealing the gold output. It detects 64.7% of 563 semantic mutants, identifies a substantial fraction of accidental corrects, and yields useful precision/recall tradeoffs on natural agent programs without an additional LLM verification call. The remaining false positives and undetected aggregation errors reinforce the intended interpretation: metamorphic execution is a complementary runtime verifier, not a correctness certificate.

References

Steven Cho, Stefano Ruberto, and Valerio Terragni. LLMORPH: Automated metamorphic testing of large language models. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*, 2025. doi: 10.1109/ASE63991.2025.00385.

Markus Dücker, Vaibhav Kumar, Yi Liu, Ronak Chaudhary, Andreas Plesner, Francisco Guzmán, and Anish Athalye. Verifying agents in rubric-graded environments. In *KDD 2026 Workshop on*

Agentic AI Evaluation and Trustworthiness, 2026. URL https://openreview.net/forum?id=5YK4zLxBfG.

Xueyu Hu, Ziyu Zhao, Shuang Wei, Ziwei Chai, Qianli Ma, Guoyin Wang, Xuwu Wang, Jing Su, Jingjing Xu, Ming Zhu, Yao Cheng, Jianbo Yuan, Jiwei Li, Kun Kuang, Yang Yang, Hongxia Yang, and Fei Wu. InfiAgent-DABench: Evaluating agents on data analysis tasks. *arXiv preprint arXiv:2401.05507*, 2024.

Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Scott Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. DS-1000: A natural and reliable benchmark for data science code generation. *arXiv preprint arXiv:2211.11501*, 2022.

Boyan Li, Zhuowen Liang, Yupeng Xie, Xiaotian Lin, Tianqi Luo, Xinyu Liu, Yizhang Zhu, Zhangyang Peng, Yuan Li, Zhengxuan Zhang, Jiayi Zhang, Nan Tang, Guoliang Li, and Yuyu Luo. DataSpace: Benchmarking data agents for verifiable analytics over heterogeneous workspaces. *arXiv preprint arXiv:2608.03451*, 2026.

Pingchuan Ma and Shuai Wang. MT-Teql: Evaluating and augmenting consistency of text-to-SQL models with metamorphic testing. *arXiv preprint arXiv:2012.11163*, 2020.

Zeyao Ma, Jing Zhang, Xiaokang Zhang, Jiaxi Yang, Zongmeng Zhang, Jiajun Zhang, Yuheng Jing, Lei Zhang, Hao Zheng, Wenting Zhao, Junyang Lin, and Binyuan Hui. Scaling agentic verifier for competitive coding. *arXiv preprint arXiv:2602.04254*, 2026. doi: 10.48550/arXiv.2602.04254.

Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Raj Sinha, Jinwoo Shin, and Tomas Pfister. DS-STAR: Data science agent for solving diverse tasks across heterogeneous formats and open-ended queries. *arXiv preprint arXiv:2509.21825*, 2026.

Shuyin Ouyang, Dong Huang, Jingwen Guo, Zeyu Sun, Qihao Zhu, and Jie M. Zhang. DSCodeBench: A realistic benchmark for data science code generation. *arXiv preprint arXiv:2505.15621*, 2025.

Goran Petrović, Marko Ivanković, Gordon Fraser, and René Just. Practical mutation testing at scale. *arXiv preprint arXiv:2102.11378*, 2021.

Mizanur Rahman, Mohammed Saidul Islam, Ridwan Mahbub, Md Tahmid Rahman Laskar, Shafiq Joty, and Enamul Hoque Prince. DSAgentBench: Can agents automate end-to-end data-science workflows in real computer environments? *arXiv preprint arXiv:2608.10366*, 2026.

Sergio Segura, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortes. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016. doi: 10.1109/TSE.2016.2532875.

Zhaoyan Sun, Shan Zhong, Daizhou Wen, Jiaxing Han, Guoliang Li, Ying Yan, Peng Zhang, Yu Su, Xiang Qi, Baolin Sun, Chengyuan Yang, Tao Fang, and Huaiyu Ruan. AgenticDataBench: A comprehensive benchmark for data agents. *arXiv preprint arXiv:2607.01647*, 2026.

Xiaoyin Wang and Dakai Zhu. Validating LLM-generated programs with metamorphic prompt testing. *arXiv preprint arXiv:2406.06864*, 2024. doi: 10.48550/arXiv.2406.06864.

Bo Yang, Yinfen Xia, Weisong Sun, and Yang Liu. Hallucination detection for LLM-based text-to-SQL generation via two-stage metamorphic testing. *arXiv preprint arXiv:2512.22250*, 2025.

Dan Zhang, Sining Zhoubian, Min Cai, Fengzu Li, Lekang Yang, Wei Wang, Tianjiao Dong, Ziniu Hu, Jie Tang, and Yisong Yue. DataSciBench: An LLM agent benchmark for data science. *arXiv preprint arXiv:2502.13897*, 2025.

A Additional Experimental Detail

A.1 Condition A on 101 tasks (full task set)

For reference, the following reports Condition A (naive seed-42 labels) on all 101 tasks, including the 3 filter-non-discriminating tasks excluded from the main 98-task evaluation set. This result is not directly comparable to Conditions B/C because task membership differs.

Table 4: Condition A naive metrics on all 101 tasks. Not directly comparable to B/C (different task set).

Model	Precision	Recall	FPR	F1	AAR
Haiku 4.5	58.8	79.0	34.4	67.4	16.7
Sonnet 4.6	66.7	72.2	20.0	69.3	16.1
Opus 4.5	58.8	81.1	33.9	68.2	14.6

A.2 Condition-B confusion matrices

Table 5: Cross-seed corrected confusion matrices. Original execution failures are excluded.

Model	TP	FP	TN	FN	Evaluated
Haiku 4.5	42	6	37	11	96
Sonnet 4.6	33	4	45	16	98
Opus 4.5	38	10	38	10	96

A.3 Seed-42 execution breakdown

Table 6: Original seed-42 program outcomes on the 98-task filter-discriminating set.

Model	Correct	Wrong executable	Execution failure
Haiku 4.5	59	37	2
Sonnet 4.6	62	36	0
Opus 4.5	61	35	2

A.4 Apparent false positives under cross-seed labels

We treat cross-seed labels as conservative operational labels rather than semantic proof. Every verifier flag on a program that passes the evaluated seeds is counted as a false positive, even when manual inspection suggests imprecise logic that may fail on an untested distribution. We retain this conservative convention to avoid post-hoc relabeling.

A.5 Mutant-generation funnel

The verified funnel is $3{,}030 \rightarrow 709 \rightarrow 563$: generated candidates, executable candidates entering equivalence evaluation, and retained non-equivalent mutants, respectively. Of the 709 executable candidates, 146 (20.6%) were equivalent across five oracle seeds and removed. An intermediate `rulemut_stats.json` file from an earlier run is not used for final results.

A.6 Repair pairing and provenance

The repair log contains 364 rows: 91 source programs crossed with four strategies. Each row includes a stable program identifier, model, data seed, and source hash, enabling row-paired comparisons. The source-program distribution is 25 programs from seed 7, 38 from seed 42, and 28 from seed 123.

Table 7: Condition-B apparent false positives by dominant relation. Some MR-F1 flags may reflect latent semantic errors not exposed by the two held-out seeds.

Relation	Haiku	Sonnet	Opus	Typical cause
MR-F1	4	2	6	Filter logic imprecision under extreme-row injection
MR-G3	1	1	2	Tie-break sensitivity
MR-H1	1	1	2	Legal computed constants

A.7 Reproducibility

Task specifications, data generators, relation implementations, evaluation scripts, aggregate result tables, and de-identified logs are included in the supplementary material. Author-identifying repository links are omitted during double-blind review.